

unit-3

CCE

* title:- music playlist manager using doubly linked list with undo and redo functionality.

* problem statement:-

music playlist manager using doubly linked list.

A music player needs to maintain a playlist of songs such as song ID, song name and artist name. Since songs may be added, deleted, or rearranged at any time a dynamic data structure is required to manage the playlist efficiently. The problem is the design and implementation of a music playlist manager using a doubly linked list. Each node stores the details of one song along with two pointers:- previous and next, which connect it to the preceding and succeeding songs.

The system should allow users to perform basic playlist management operations such as adding new song ID, song name, displaying all songs, searching for a song, deleting a song, rearranging song and moving in both forward & reverse direction. It should also support undo & redo operations to store previous playlist changes & apply undone actions. The implementation should maintain the links correctly after every modification & demonstrate dynamic memory allocation, doubly linked list traversal & stack based history.

* Objective.

- To store song details such as Song ID, Song Name & Artist Name.
- To perform insertion and deletion of song dynamically.
- To display song in both Forward & Reverse direction.
- To search for song using its ID.
- To rearrange ~~music~~ songs within the playlist.
- To navigate to the next & previous songs.
- To implement undo & redo & next pointers after every modification.
- To demonstrate dynamic memory allocation & linked list operations in C++.

* Algorithm.

Step 1:- Start the program.

Step 2:- Initialize the doubly linked list with head, tail and current pointer set to Null. (14)

Step 3:- Display to main menu containing following operations:

- Add song
- Display Playlist Forward.
- Display Playlist Reverse
- Search song
- Delete song
- Rearrange song
- Next song
- Previous song

- undo
- Rendo
- Exit.

Step 4:- Read the user's choice

Steps:- Perform the selected operation.

- Add song:- Read song details, create a new node using dynamic memory allocation and insert it at the end of the playlist.
 - Display Forward:- Traverse from head to tail using the next pointer.
 - Display reverse:- Traverse from tail to head using the prev pointer.
 - Search song:- Search for the song using its ID & display its details & delete it if found.
 - Delete song:- Find the song, update the neighboring pointers & delete its memory.
 - Rearrange song:- Remove the selected song from its current position & insert it at the specific position.
 - Next song:- move the current pointer to the next song.
 - Previous song:- move the current pointer to the previous song.
 - undo:- Restore the most recently undone playlist state using the undo stack.
 - Rendo:- Restore the most recently undone playlist state using the redo stack.
- Steps:- Respect the menu until the user selects exit.

Step 7:- Free the dynamically allocated memory and clear the playlist.

steps: - stop the program.

* Flowchart.

